EXPERIMENT NO. 6

Introduction to MPI

Installation, Processes, Ranks, Groups and Point-to-Point Communication

HPC LAB

Student Details	Value
Name	Gaurav Madhav Patil
PRN	23510040
Batch	B6
Experiment	6

1. Objective

To configure MPI and understand distributed-memory parallel programming by creating processes, identifying ranks, working with groups and communicators, and performing one-to-one communication using MPI_Send() and MPI_Recv().

2. Background / Use

Message Passing Interface (MPI) is a programming model and standard used for distributed-memory parallel computing. An MPI program can create multiple processes, where each process has its own memory space. Processes communicate by explicitly exchanging messages. MPI provides concepts such as ranks, communicators and groups to organize processes and communication.

3. Section A — Task 1: MPI Installation and Environment Configuration

Open MPI was installed and verified successfully. The following commands were used to identify the MPI implementation, compiler, operating system and CPU configuration.

Parameter	Measured / Detected Value
MPI implementation	Open MPI
MPI version	4.1.2
C compiler through mpicc	GCC 11.4.0
Operating system	Ubuntu 22.04.5 LTS (Jammy)
Architecture	x86_64
CPU	12th Gen Intel(R) Core(TM) i5-12450H
CPU configuration	8 cores / 12 logical CPUs, 1 socket
NUMA nodes	1

Verification Commands

```
mpicc --version
mpirun --version
lsb_release -a
lscpu
```

Execution Output

```
mpicc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0

mpirun --version
mpirun (Open MPI) 4.1.2

lsb_release -a
Distributor ID: Ubuntu
Description: Ubuntu
Release: 22.04
Codename: jammy
```

The lscpu command showed an x86_64 system with an Intel Core i5-12450H, 8 physical cores and 12 logical CPUs. MPI commands were available and working correctly.

4. Section A — Task 2: First MPI Program

The first MPI program initializes MPI, prints “Hello from MPI” from every process, and then finalizes MPI.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    printf("Hello from MPI\n");

    MPI_Finalize();

    return 0;
}
```

File: hello.c

Compilation command:

```
mpicc hello.c -o hello
```

Execution command:

```
mpirun -np 4 ./hello
```

Execution Output

```
Hello from MPI
Hello from MPI
Hello from MPI
Hello from MPI
```

Result: The program executed successfully with 4 MPI processes.

5. Section A — Task 3: Process Rank and Number of Processes

MPI_Comm_rank() was used to obtain the rank of each process and MPI_Comm_size() was used to obtain the total number of processes in MPI_COMM_WORLD.

```
#include <mpi.h>
#include <stdio.h>
```

```
int main(int argc, char *argv[])
{
    int rank, size;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Process %d of %d\n", rank, size);

    MPI_Finalize();

    return 0;
}
```

File: rank_size.c

```
mpicc rank_size.c -o rank_size
```

Execution with 2 processes:

```
mpirun -np 2 ./rank_size
```

Execution Output

```
Process 0 of 2
Process 1 of 2
```

Execution with 4 processes:

```
mpirun -np 4 ./rank_size
```

Execution Output

```
Process 0 of 4
Process 3 of 4
Process 2 of 4
Process 1 of 4
```

Execution with 8 processes:

```
mpirun -np 8 ./rank_size
```

Execution Output

```
Process 2 of 8
Process 6 of 8
Process 4 of 8
Process 1 of 8
Process 3 of 8
Process 5 of 8
Process 7 of 8
Process 0 of 8
```

Observation: All required ranks were created and the reported process count matched the requested number of processes. The order of output was not fixed; MPI processes may execute and print in different orders because they run concurrently.

6. Section A — Task 4: Groups and Communicators

MPI_COMM_WORLD contains all MPI processes. Its associated MPI group was obtained using MPI_Comm_group(). A new group containing ranks 0, 2, 4 and 6 was created using MPI_Group_incl(), and a communicator for that group was created using MPI_Comm_create_group().

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int world_rank;
    MPI_Group world_group, new_group;
    MPI_Comm new_comm;

    int ranks[] = {0, 2, 4, 6};

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    MPI_Comm_group(MPI_COMM_WORLD, &world_group);

    MPI_Group_incl(world_group, 4, ranks, &new_group);

    MPI_Comm_create_group(MPI_COMM_WORLD, new_group, 0, &new_comm);

    if (new_comm != MPI_COMM_NULL)
    {
        int new_rank;
        MPI_Comm_rank(new_comm, &new_rank);

        printf("Global rank %d -> New communicator rank %d\n",
               world_rank, new_rank);

        MPI_Comm_free(&new_comm);
    }

    MPI_Group_free(&new_group);
    MPI_Group_free(&world_group);

    MPI_Finalize();

    return 0;
}
```

File: group_comm.c

```
mpicc group_comm.c -o group_comm
mpirun -np 8 ./group_comm
```

Execution Output

```
Global rank 0 -> New communicator rank 0
Global rank 2 -> New communicator rank 1
Global rank 4 -> New communicator rank 2
Global rank 6 -> New communicator rank 3
```

Global Rank	New Communicator Rank
0	0
2	1
4	2
6	3

Observation: The global rank identifies a process in MPI_COMM_WORLD, while the communicator-local rank identifies the same process within the newly created communicator. Ranks 1, 3, 5 and 7 were not members of the selected communicator and therefore produced no communicator-rank output.

HPC Programmer Questions

1. A process can belong to more than one communicator because different communicators can define different groups of processes. Its rank is relative to the communicator being used.
2. Communicators are important because they define the communication context and the set of processes that participate in that communication. They help keep separate communication activities logically independent.

7. Section B — Task 5: Point-to-Point String Communication

Process 0 sends a character message to Process 1 using MPI_Send(), and Process 1 receives it using MPI_Recv().

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank;
    char message[] = "Hello from Process 0";
    char received[100];

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0)
    {
        MPI_Send(message, 100, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1)
    {
        MPI_Recv(received, 100, MPI_CHAR, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process 1 received: %s\n", received);
    }

    MPI_Finalize();

    return 0;
}
```

File: send_receive.c

```
mpicc send_receive.c -o send_receive
mpirun -np 2 ./send_receive
```

Execution Output

```
Process 1 received: Hello from Process 0
```

Result: The message was transferred successfully from rank 0 to rank 1.

8. Section B — Task 6: Integer Communication

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank;
```

```
    int value = 100;
    int received;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0)
    {
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1)
    {
        MPI_Recv(&received, 1, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process 1 received value = %d\n", received);
    }

    MPI_Finalize();

    return 0;
}
```

File: integer.c

```
mpicc integer.c -o integer
mpirun -np 2 ./integer
```

Execution Output

```
Process 1 received value = 100
```

Result: The integer value 100 was received correctly by Process 1.

9. Section B — Task 7: Integer Array Communication

Process 0 sends the array 10, 20, 30, 40, 50 to Process 1. Process 1 computes the sum, average and maximum.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank;
    int array[5] = {10, 20, 30, 40, 50};
    int received[5];

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0)
    {
        MPI_Send(array, 5, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1)
    {
        MPI_Recv(received, 5, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        int sum = 0;
```

```
        int max = received[0];

        for (int i = 0; i < 5; i++)
        {
            sum += received[i];

            if (received[i] > max)
                max = received[i];
        }

        double average = sum / 5.0;

        printf("Received array: ");
        for (int i = 0; i < 5; i++)
            printf("%d ", received[i]);

        printf("\nSum = %d\n", sum);
        printf("Average = %.2f\n", average);
        printf("Maximum = %d\n", max);
    }

    MPI_Finalize();

    return 0;
}
```

File: array.c

```
mpicc array.c -o array
mpirun -np 2 ./array
```

Execution Output

```
Received array: 10 20 30 40 50
Sum = 150
Average = 30.00
Maximum = 50
```

Result: The array was transferred correctly and all calculations matched the expected values.

10. Section B — Performance Activity: Communication Time

Communication time was measured for messages of 1 KB, 10 KB, 100 KB, 1 MB and 10 MB using MPI_Wtime(). Process 0 measured the time taken by the blocking MPI_Send() operation.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int rank;
    int sizes[] = {1024, 10 * 1024, 100 * 1024,
                   1024 * 1024, 10 * 1024 * 1024};
    int num_sizes = 5;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    for (int i = 0; i < num_sizes; i++)
    {
        int size = sizes[i];
        char *buffer = malloc(size);
```

```
        if (rank == 0)
        {
            double start = MPI_Wtime();

            MPI_Send(buffer, size, MPI_CHAR, 1, 0, MPI_COMM_WORLD);

            double end = MPI_Wtime();

            printf("Message Size: %d bytes, Communication Time: %.9f seconds\n",
                   size, end - start);
        }
        else if (rank == 1)
        {
            MPI_Recv(buffer, size, MPI_CHAR, 0, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }

        free(buffer);

        MPI_Barrier(MPI_COMM_WORLD);
    }

    MPI_Finalize();

    return 0;
}
```

File: performance.c

```
mpicc performance.c -o performance
mpirun -np 2 ./performance
```

Execution Output

```
Message Size: 1024 bytes, Communication Time: 0.000026269 seconds
Message Size: 10240 bytes, Communication Time: 0.000034650 seconds
Message Size: 102400 bytes, Communication Time: 0.000064147 seconds
Message Size: 1048576 bytes, Communication Time: 0.000598991 seconds
Message Size: 10485760 bytes, Communication Time: 0.003845994 seconds
```

Message Size	Communication Time (s)
1 KB	0.000026269
10 KB	0.000034650
100 KB	0.000064147
1 MB	0.000598991
10 MB	0.003845994

Observation: The measured communication time increased as the message size increased. Larger messages require more data to be transferred, so the communication overhead becomes more significant.

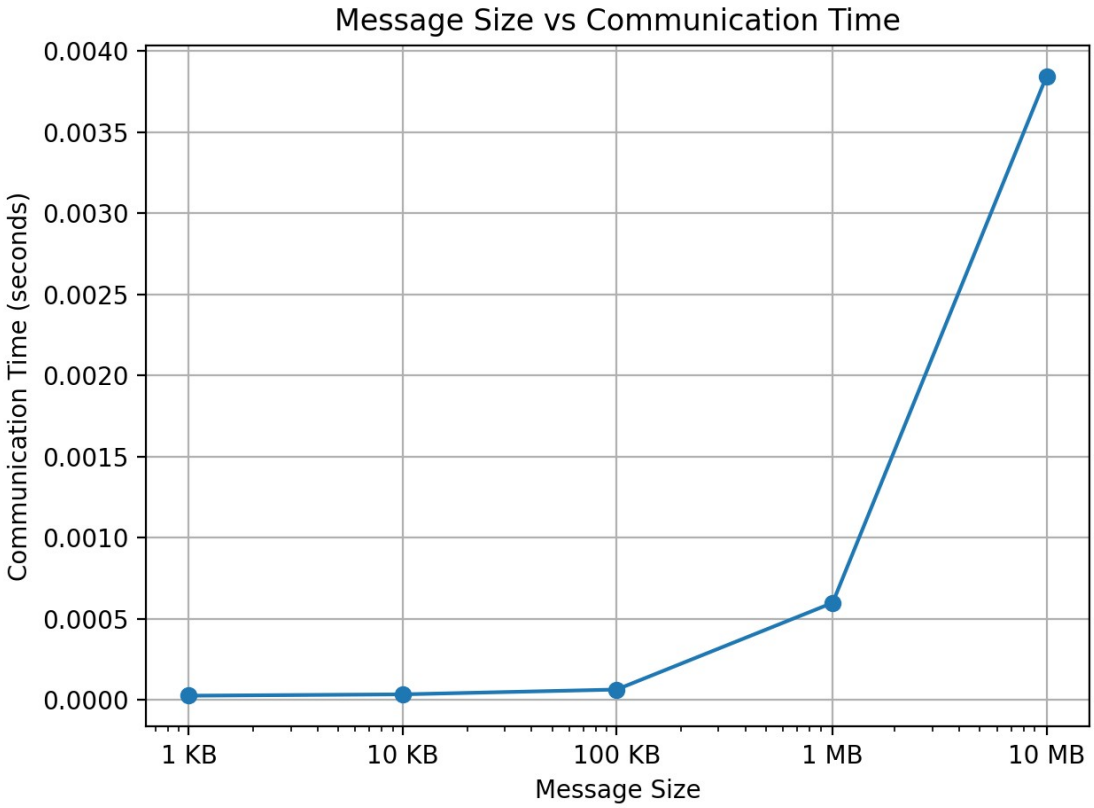


Figure 1: Message Size vs Communication Time

11. Section B — Challenge: Ping-Pong Communication and RTT

In the ping-pong experiment, Process 0 sends a message to Process 1 and Process 1 sends the message back. This exchange is repeated for 1000 iterations. Average round-trip time (RTT) is calculated as Total Time / Iterations.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int rank;
    int iterations = 1000;
    int sizes[] = {
        1024,
        10 * 1024,
        100 * 1024,
        1024 * 1024,
        10 * 1024 * 1024
    };

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    for (int s = 0; s < 5; s++)
    {
```

```
        int message_size = sizes[s];
        char *buffer = malloc(message_size);

        MPI_Barrier(MPI_COMM_WORLD);

        double start = MPI_Wtime();

        for (int i = 0; i < iterations; i++)
        {
            if (rank == 0)
            {
                MPI_Send(buffer, message_size, MPI_CHAR, 1, 0,
                         MPI_COMM_WORLD);

                MPI_Recv(buffer, message_size, MPI_CHAR, 1, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            }
            else if (rank == 1)
            {
                MPI_Recv(buffer, message_size, MPI_CHAR, 0, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);

                MPI_Send(buffer, message_size, MPI_CHAR, 0, 0,
                         MPI_COMM_WORLD);
            }
        }

        double end = MPI_Wtime();

        if (rank == 0)
        {
            double total_time = end - start;
            double rtt = total_time / iterations;

            printf("%d bytes | %d iterations | Total: %.9f s | RTT: %.9f s\n",
                   message_size, iterations, total_time, rtt);
        }

        free(buffer);
    }

    MPI_Finalize();

    return 0;
}
```

File: ping_pong_all.c

```
mpicc ping_pong_all.c -o ping_pong_all
mpirun -np 2 ./ping_pong_all
```

Execution Output

```
1024 bytes | 1000 iterations | Total: 0.000743978 s | RTT: 0.000000744 s
10240 bytes | 1000 iterations | Total: 0.004450464 s | RTT: 0.000004450 s
102400 bytes | 1000 iterations | Total: 0.022206427 s | RTT: 0.000022206 s
1048576 bytes | 1000 iterations | Total: 0.115214898 s | RTT: 0.000115215 s
10485760 bytes | 1000 iterations | Total: 2.170387654 s | RTT: 0.002170388 s
```

Message Size	Iterations	Total Time (s)	Average RTT (s)
1 KB	1000	0.000743978	0.000000744
10 KB	1000	0.004450464	0.000004450
100 KB	1000	0.022206427	0.000022206
1 MB	1000	0.115214898	0.000115215
10 MB	1000	2.170387654	0.002170388

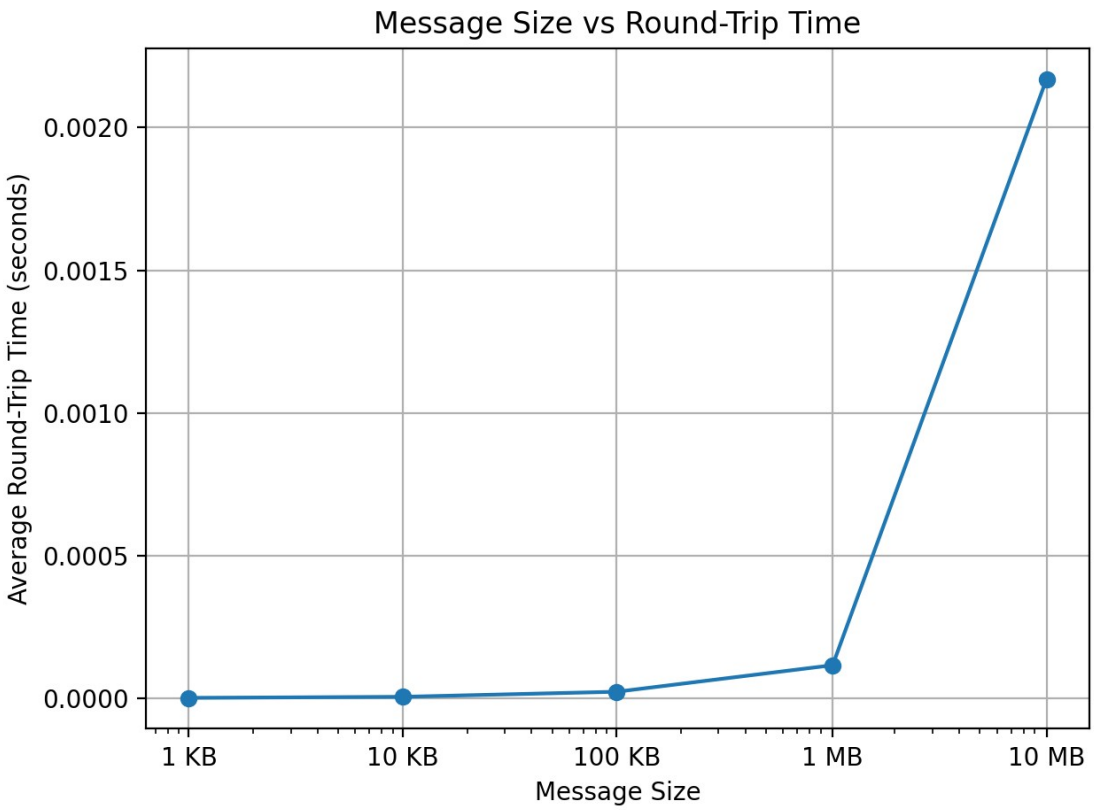


Figure 2: Message Size vs Round-Trip Time (RTT)

Analysis: RTT increased with message size. The smallest message had a very small RTT, while the 10 MB message had the highest RTT. This demonstrates that communication cost becomes increasingly important for large messages. The relationship is not perfectly linear because real execution also includes MPI protocol, buffering, synchronization and operating-system effects.

12. Section C — Advanced Challenge: Blocking Communication and Deadlock

Blocking communication can lead to deadlock if processes wait for communication operations that cannot complete. For example, if both processes perform blocking MPI_Send() operations first and neither reaches the matching MPI_Recv(), both processes may wait indefinitely depending on the MPI implementation and message size.

A safe ordering was implemented instead: Process 0 sends and then receives, while Process 1 receives and then sends. This avoids the circular wait.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank;
    int value = 100;
    int received;
```

```
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0)
    {
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&received, 1, MPI_INT, 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process 0 completed send and receive.\n");
    }
    else if (rank == 1)
    {
        MPI_Recv(&received, 1, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Send(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);

        printf("Process 1 completed receive and send.\n");
    }

    MPI_Finalize();

    return 0;
}
```

File: deadlock_demo.c

```
mpicc deadlock_demo.c -o deadlock_demo
mpirun -np 2 ./deadlock_demo
```

Execution Output

```
Process 1 completed receive and send.
Process 0 completed send and receive.
```

The intentionally deadlocking version was not run as an unbounded experiment. The controlled safe version demonstrated the correct ordering of blocking operations.

13. Section D — AI/HPC Case Study: Distributed Deep Learning

Distributed deep learning is an important application of distributed computing because training large neural network models can require a large amount of computational power, memory and training data. A single computer may not be sufficient to process a large dataset or train a complex model within a reasonable amount of time. Multiple computing nodes can therefore work together to perform the training process.

In a distributed deep learning system, each MPI process can be assigned a portion of the training data or a specific computational task. For example, different processes can process different batches of training data simultaneously. Each process performs computations using its assigned data and produces results such as gradients or intermediate values.

Communication is required because the processes need to exchange information during training. In a data-parallel approach, processes may exchange computed gradients so that model parameters can be updated consistently. The processes involved in the same training operation need to communicate with each other to coordinate their computations and updates.

MPI can provide the communication mechanism between these processes. MPI processes can be created on multiple computing nodes, and each process can be identified using its rank. Communicators can organize processes that need to participate in particular operations. MPI communication functions such as MPI_Send() and MPI_Recv() can be used when one process needs to send data to another process.

Communication performance is an important factor in distributed deep learning. If communication takes longer than computation, processes may spend significant amounts of time waiting for data from other processes. In such a situation, increasing the number of processes may not provide proportional performance improvement. Therefore, an efficient distributed application should balance computation and communication and avoid unnecessary data transfers.

This demonstrates why MPI is useful for large-scale applications: it allows independent processes running in separate memory spaces to cooperate through message passing.

14. Questions — Basic

1. What is MPI?

MPI (Message Passing Interface) is a standard programming model for distributed-memory parallel computing. It allows multiple processes to communicate by exchanging messages.

2. What is an MPI process?

An MPI process is an independent instance of an MPI program. Each process has its own memory space and is assigned a rank within a communicator.

3. What is an MPI rank?

An MPI rank is a unique integer identifier assigned to a process within a communicator. It is used to identify the process during communication.

4. What is MPI_COMM_WORLD?

MPI_COMM_WORLD is the default communicator created by MPI. It contains all processes participating in the MPI program.

5. What is the purpose of MPI_Send() and MPI_Recv()?

MPI_Send() sends data from one MPI process to another. MPI_Recv() receives data sent by another MPI process.

15. Questions — Moderate

6. Differentiate between an MPI group and an MPI communicator.

An MPI group is a collection of process identities. A communicator associates a group with a communication context and is used by MPI communication operations.

7. Can the same process have different ranks in different communicators? Explain.

Yes. Rank values are relative to a communicator. The same process can have one rank in MPI_COMM_WORLD and a different rank in another communicator containing a subset of processes.

8. What information is specified in MPI_Send()?
MPI_Send() specifies the data buffer, number of elements, datatype, destination rank, message tag and communicator.

9. What information is specified in MPI_Recv()?
MPI_Recv() specifies the receive buffer, maximum number of elements, datatype, source rank, message tag, communicator and status information.

10. Why does message ordering matter in MPI?
Message ordering matters because sends and receives must match appropriately by source, destination, tag and communicator. Incorrect ordering can cause unexpected behavior or blocking/deadlock.

16. Questions — Advanced

11. What is communication overhead in MPI?
Communication overhead is the time and system resources spent exchanging data and coordinating processes instead of performing useful computation.

12. How does message size affect communication time?
The measured results showed that communication time generally increased as message size increased. Larger messages require more data to be transferred and therefore have greater communication cost.

13. What is deadlock?
Deadlock is a situation in which processes wait indefinitely for operations that cannot complete, so program execution cannot proceed.

14. How can blocking communication cause deadlock?
If two processes perform blocking operations in an order that makes each process wait for the other to reach a matching operation, neither process can progress. A safe communication order or suitable nonblocking communication can prevent this.

15. Why is communication important in distributed-memory systems?
Processes in distributed-memory systems do not automatically share their data. Communication is therefore required to exchange data and coordinate computation. Excessive communication can reduce parallel efficiency.

17. Performance Analysis
The required ping-pong measurements were collected for five message sizes. Total communication time and average RTT were recorded for 1000 iterations.

Message Size	Iterations	Total Time (s)	Average RTT (s)
1 KB	1000	0.000743978	0.000000744
10 KB	1000	0.004450464	0.000004450
100 KB	1000	0.022206427	0.000022206
1 MB	1000	0.115214898	0.000115215
10 MB	1000	2.170387654	0.002170388

Trend discussion: Both communication time and RTT increased as message size increased. For small messages, fixed communication overhead is significant. As the message becomes larger, the amount of data

transferred dominates more of the communication time. Therefore, communication can become a bottleneck in distributed applications when large amounts of data must be exchanged frequently.

18. Final Questions

1. What is the fundamental difference between OpenMP and MPI?

OpenMP primarily uses multiple threads within a shared-memory system, while MPI uses multiple processes with separate memory spaces and explicit message passing.

2. When would you choose MPI over OpenMP?

MPI is preferred when an application needs to run across multiple computers or nodes, or when the target system uses distributed memory. OpenMP is more natural for shared-memory parallelism within a node.

3. Why is data distribution important in MPI?

Data distribution assigns work and data among processes. Good distribution balances the workload and reduces unnecessary communication, which improves scalability and performance.

4. What is the role of communication in MPI?

Communication allows processes with separate memory spaces to exchange the data required for their computations and coordinate their execution.

5. What challenges arise as the number of MPI processes increases?

Increasing the process count can increase communication and synchronization overhead, create load-balancing issues, and make coordination and scalability more difficult.

6. How can the concepts from this experiment be applied to the next matrix computation assignment?

A matrix can be divided among MPI processes, with each process computing on its local portion. Processes can exchange required matrix data using MPI communication. Groups and communicators can also organize subsets of processes for particular stages of the computation.

19. Conclusion

The experiment demonstrated the basic concepts of MPI for distributed-memory parallel computing. MPI was successfully configured using Open MPI 4.1.2, and programs were executed with multiple processes. The practical work demonstrated MPI initialization, process ranks, process size, groups, communicators and point-to-point communication using MPI_Send() and MPI_Recv().

String, integer and array communication were successfully performed between MPI processes. The group and communicator experiment showed the difference between a process's global rank and its rank inside a selected communicator. The performance experiment showed that communication time increased with message size, while the ping-pong experiment showed the same trend for average RTT. The blocking communication challenge also demonstrated the importance of correct message ordering to avoid deadlock.

Overall, the experiment provides the foundation for larger distributed-memory HPC applications. The concepts of process decomposition, rank identification, communicator-based organization and communication will be useful when implementing distributed matrix computations and other large-scale parallel applications.

20. Programs Completed

Program	Purpose
hello.c	First MPI program — Hello from MPI
rank_size.c	Display process rank and total number of processes
group_comm.c	Create a subgroup and communicator
send_receive.c	Send and receive a string
integer.c	Send and receive an integer
array.c	Send an integer array and compute statistics
performance.c	Measure communication time for different message sizes
ping_pong.c	Basic ping-pong communication for 1 KB
ping_pong_all.c	Ping-pong communication for all required message sizes
deadlock_demo.c	Controlled safe demonstration of blocking communication ordering